

1. Difference Between Closed Source, Open Source, and Open Weight Models

Closed Source Models

These models are developed by companies and only accessible through APIs or paid subscriptions.

Examples: GPT-5 (OpenAI), Claude (Anthropic), Gemini (Google)

Advantages	Disadvantages
Best performance Regular updates No infrastructure management Strong safety features	Cannot inspect the model Cannot retrain the base model Usage costs money Vendor lock-in

Open-Source Models

Everything is publicly available: Source code, Model weights, Architecture, often training methodology. You can modify and train the model yourself.

Examples: GPT-NeoX, BLOOM, OLMo

Advantages	Disadvantages
Complete transparency Full control Research friendly Customisable	Requires significant hardware Maintenance responsibility Large storage requirements

Open Weight Models

This is currently the most common category. The company releases the trained weights but **not** the complete training pipeline or dataset. You can: Run locally, Fine-tune, Quantise, Build applications

Examples: Llama 3, Gemma, Mistral, Qwen, Phi

Advantages	Disadvantages
Free to run locally Fine-tune possible Excellent for AI applications	Cannot completely reproduce the original model Training data usually undisclosed

Quick Analogy

Imagine a car.

Closed Source

You can drive it.

You cannot open the engine.

Open Weight

You receive the engine.

You can improve or replace parts.

You don't know how the engine was originally manufactured.

Open Source

You receive the complete blueprints.

You know exactly how every part was built.

You can build your own version from scratch.

2. Top 3 Tools for Model Training and Fine-Tuning

I. Unsloth

Best for beginners and developers with consumer GPUs.

Features

- Extremely fast fine-tuning
- Lower VRAM usage
- Supports LoRA and QLoRA
- Hugging Face compatible
- Works well on RTX 3060/4060/4070 and cloud GPUs

Best For

- Fine-tuning Llama
- Gemma
- Qwen
- Mistral
- Phi

Pros

- Easy to learn
- Very fast
- Excellent documentation
- Active community

Cons

- Focused on fine-tuning rather than training from scratch

II. Hugging Face Transformers + PEFT

The industry standard.

Features

- Largest ecosystem
- Thousands of models
- Supports training, inference, evaluation and deployment

Pros

- Production ready
- Huge community
- Flexible

Cons

- Steeper learning curve
- More boilerplate than Unsloth

III. Axolotl

Popular among researchers and advanced practitioners.

Features

- Fine-tunes large models efficiently
- YAML-based configuration
- Multi-GPU support
- LoRA and QLoRA support

Pros

- Powerful
- Scalable
- Widely used for advanced projects

Cons

- More complex than Unsloth

3. Difference Between LLM and SLM

LLM = Large Language Model

Typically contains billions to hundreds of billions of parameters.

Examples:

- GPT-5
- Llama 3 70B
- Gemini
- Claude

LLMs are trained on enormous datasets using clusters of powerful GPUs over weeks or months.

SLM = Small Language Model

Usually contains millions to a few billion parameters.

Examples:

- Phi-3 Mini
- Gemma 3 1B/4B
- Qwen 2.5 3B
- TinyLlama

SLMs are designed to be efficient and can often run on laptops or edge devices.

Feature	LLM	SLM
Parameters	7B–1000B+	100M–7B (roughly)
Hardware	Multi-GPU clusters	Laptop or single GPU
Speed	Slower	Faster
Cost	High	Low
Memory	Large	Small
Local Execution	Difficult	Easy

Training a modern LLM requires:

- Thousands of GPUs
- Massive datasets (terabytes)
- Huge electricity costs
- Weeks or months of computation
- Teams of researchers and engineers

This puts full pre-training beyond the reach of most individuals and small teams.

4. What are the other format models are available? What is GGUF?

GGUF stands for **GPT-Generated Unified Format**. It is a modern file format designed for efficient inference with llama.cpp and related tools.

A GGUF file includes:

- Model weights
- Tokeniser
- Configuration
- Metadata

Everything needed to run the model is packaged into a single file. AI models are stored in different file formats depending on the framework and intended runtime.

Format	Used By	Purpose
PyTorch (.pt, .pth)	PyTorch	Training and inference
Safetensors (.safetensors)	Hugging Face	Secure, fast model storage
GGUF (.gguf)	llama.cpp, LM Studio, Jan	Efficient local inference
ONNX (.onnx)	ONNX Runtime	Cross-platform deployment
TensorFlow (.ckpt, SavedModel)	TensorFlow	Training and serving
MLX	Apple MLX	Apple Silicon optimisation